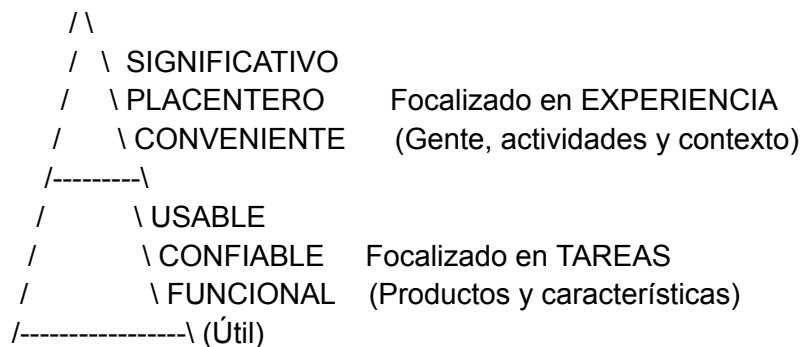


Visión y Propósito de la Creación de Productos

- **¿Por qué creamos productos?:** La creación de un producto no responde únicamente a ganar dinero, masividad o satisfacer peticiones de clientes, sino a una visión más ambiciosa de modificar la rutina diaria o impactar en el entorno.
 - **Eliminación del desperdicio:** Frecuentemente se invierte esfuerzo en construir características que los usuarios nunca o casi nunca utilizan. La gestión de productos busca eliminar este desperdicio para focalizar el desarrollo en lo que realmente aporta valor.
-

La Pirámide de Evolución de los Productos de Software

Los productos de software evolucionan atravesando distintas capas de expectativas y experiencia del usuario:



1. **Base de la Pirámide (Focalizado en Tareas):**
 - **Funcional (Útil), Confiable y Usable:** Constituyen la expectativa básica que cualquier usuario asume al utilizar un software. La mayoría de los desarrollos organizacionales se quedan estancados en este nivel.
 2. **Tope de la Pirámide (Focalizado en Experiencias):**
 - **Conveniente, Placentero y Significativo:** Representan las dimensiones vinculadas a la visión superior del producto. El desafío técnico y de gestión consiste en eliminar el desperdicio para alcanzar estos niveles sin malgastar recursos.
-

Propuesta de Valor (UVP) y Validación con el MVP

- **UVP (User Value Proposition):** Es la técnica centrada en entender las necesidades, deseos y problemas del usuario para diseñar soluciones efectivas que resuelvan una hipótesis única.
- **El riesgo de construir a ciegas:** Crear un producto completo sin validar la hipótesis inicial genera el riesgo de malgastar recursos en algo que el mercado no demanda.

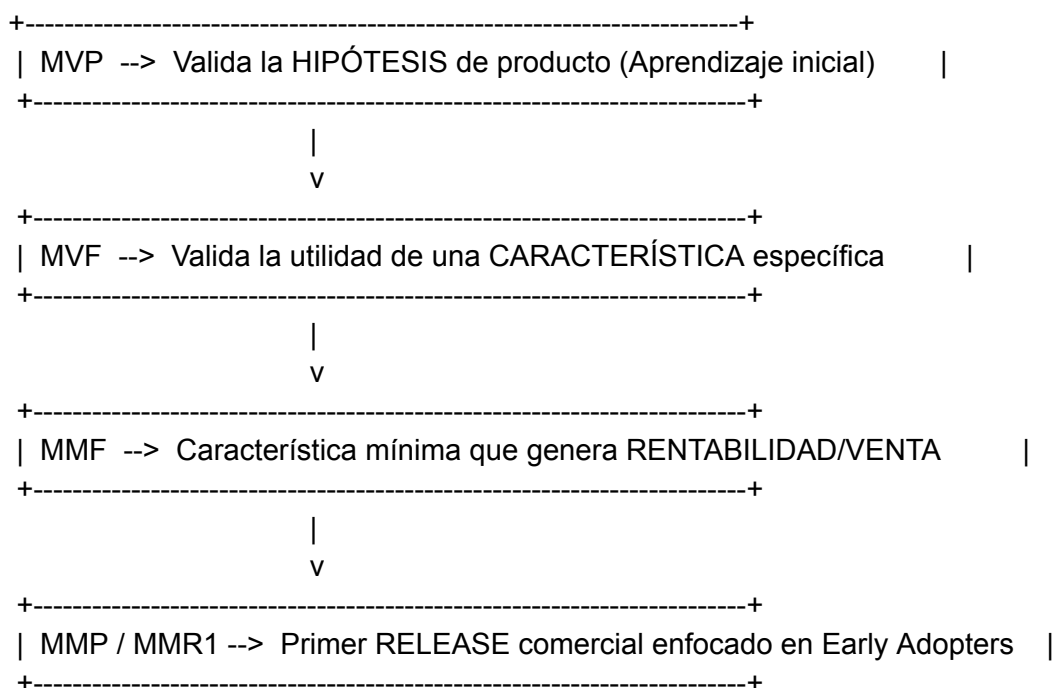
- **MVP (*Minimum Viable Product*):**
 - **Definición:** Es la versión mínima de un producto necesaria para que los potenciales usuarios lo evalúen y confirmen si la solución les atrae.
 - **Objetivo principal:** Su propósito no es vender, sino **validar la hipótesis de negocio** e iniciar el circuito de aprendizaje.

Escenarios de Retroalimentación del MVP

Feedback del Usuario	Diagnóstico	Acción a Tomar
Señal dispersa	Cada cliente solicita funcionalidades totalmente distintas.	El mercado no está claro; se debe seguir afinando la visión.
Desviación de foco	La hipótesis general es correcta pero requiere ajustes en sus características.	Mover el MVP hacia el foco de interés real de los clientes.
Pivote	La hipótesis inicial falla en validar el mercado.	Redefinir el enfoque y construir un MVP2 para iniciar un nuevo ciclo de validación.

Clasificación de Conceptos Mínimos (El Espectro de Mínimos)

Para gestionar la incertidumbre y la salida al mercado, la disciplina clasifica los artefactos en diferentes niveles:



- **MVP (*Minimum Viable Product*)**: Versión de un nuevo producto creada con el menor esfuerzo posible, dirigida a un subconjunto de clientes para obtener aprendizaje validado.
 - **MVF (*Minimum Viable Feature*)**: Característica mínima para probar rápidamente la utilidad de una funcionalidad específica en el dominio de la solución. Funciona como una versión reducida del MVP focalizada en una función.
 - **MMF (*Minimum Marketable Feature*)**: Pieza más pequeña de funcionalidad liberable que aporta valor comercial directo y rentabilidad tanto a la organización como a los usuarios.
 - **MMP (*Minimum Marketable Product*)**: Primer lanzamiento comercial de un producto dirigidos a los primeros usuarios (*early adopters*). Corresponde al **MMR1**.
 - **MMR (*Minimum Marketable Release*)**: Incremento o release más pequeño que ofrece un conjunto nuevo de funcionalidades que satisface necesidades actuales.
-

Errores Frecuentes al Trabajar con Productos Mínimos

1. **Confundir MVP con MMF o MMP**: El MVP se enfoca estrictamente en **aprender y validar hipótesis**, mientras que la MMF y el MMP se orientan a **obtener retorno económico y vender**.
 2. **Priorizar lo "Mínimo" omitiendo lo "Viable"**: Entregar un producto con calidad insuficiente impide evaluar con precisión si los clientes realmente utilizarían la solución.
 3. **Estancamiento tras la entrega**: Lanzar un MVP y no realizar iteraciones ni cambios posteriores ignorando la retroalimentación recibida.
-

El Ciclo de Aprendizaje Validado (*Build-Experiment-Learn*)

El éxito del desarrollo no reside en la mera entrega de software, sino en asegurar que la característica entregada sea efectivamente utilizada.

{Idea} → {Construir} → {Producto} → {Experimentar} → {Datos} → {Aprender}

- **Fase Construir**: Ante la duda se debe simplificar, evitando la sobre construcción o la promesa excesiva de características. Todo trabajo adicional fuera de lo estrictamente necesario para aprender constituye desperdicio.
 - **Fase Experimentar**: Es el núcleo del ciclo, donde se confrontan las suposiciones con datos reales del mercado.
-

Reducción del Riesgo: "Saltos de Fe" y Preparación del MVP

- **La Audacia de Cero**: Posponer la experimentación por buscar un producto perfecto genera acumulación de trabajo desperdiciado y alto riesgo de construir algo

irrelevante. Se debe usar el MVP para experimentar en silencio desde etapas tempranas.

- **Supuestos de "Saltos de Fe" (*Leaps of Faith*):** Son las suposiciones más riesgosas sobre las cuales se sostiene el concepto de un nuevo producto o startup.
 - **Pasos para preparar un MVP:**
 1. Encontrar un nicho de mercado.
 2. Crear un *Roadmap* realista.
 3. Investigar a la competencia.
 4. Pre-vender el MVP.
 5. Testear las suposiciones.
 6. Asegurar que el MVP resuelve el problema correcto.
 7. Focalizarse en las funcionalidades principales.
-

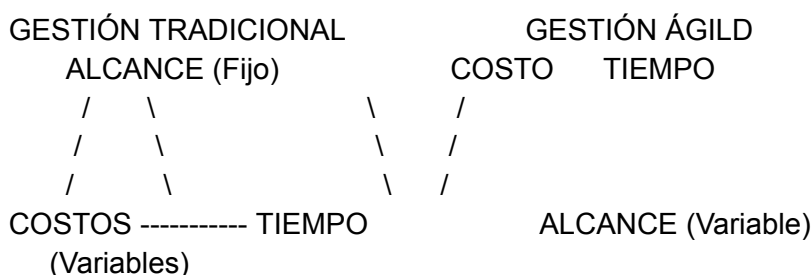
Enfoque de Requerimientos: Tradicional vs. Ágil

La gestión ágil de requerimientos nace como respuesta a las limitaciones del enfoque tradicional prescriptivo.

- **El problema del BRUF (*Big Requirements Up Front*)**: En la gestión tradicional se intenta especificar el 100% de los requerimientos en detalle antes de empezar a construir. Esto genera un **desperdicio masivo**: cerca del **45% de las funcionalidades desarrolladas NUNCA son utilizadas** por el usuario final.
 - **Principio *Just in Time* (JIT)**: En agilidad se define "**sólo lo suficiente**" para iniciar el desarrollo y obtener retroalimentación temprana. Los requerimientos se analizan en detalle en el momento adecuado (*Just in Time*), justo antes de ser implementados, evitando desperdiciar esfuerzo en especificar cosas que cambiarán.
 - **El Requerimiento como Vehículo de Valor**: El software no es el fin, sino el medio para entregar **valor de negocio**.
-

Triángulo de Hierro vs. Triángulo Ágil

La diferencia estructural entre ambos enfoques se refleja en el ajuste de sus variables principales:



- **Gestión Tradicional (Triángulo de Hierro)**: El **Alcance** se congela al inicio; por ende, los **Costos** y el **Tiempo** deben variar y ajustarse para intentar cumplir lo pactado.
 - **Gestión Ágil (Triángulo Invertido)**: El **Tiempo** (duración del Sprint) y los **Costos** (equipo fijo) permanecen **fijos**. La variable de ajuste es el **Alcance**, el cual se prioriza continuamente según el valor entregado.
-

Niveles y Tipos de Requerimientos

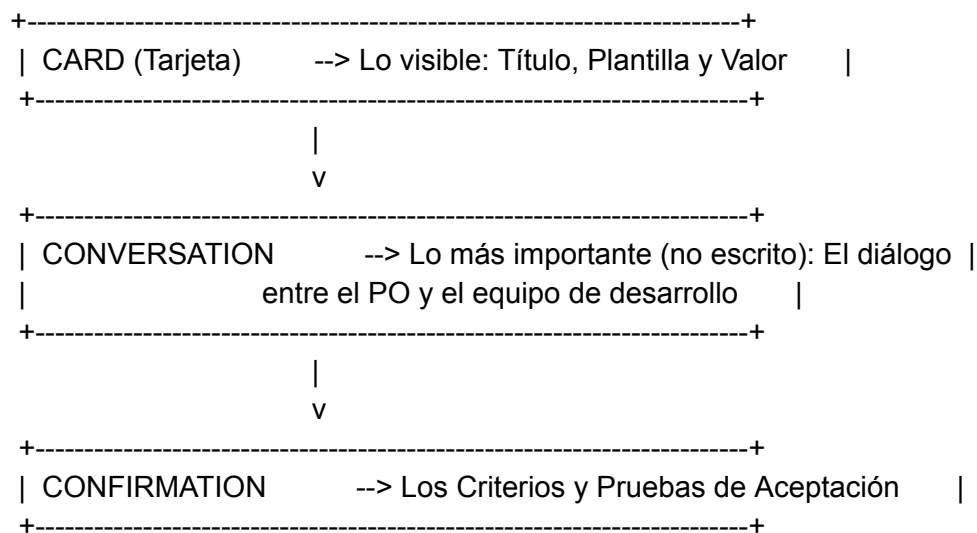
La cátedra clasifica los requerimientos para diferenciar el **Dominio del Problema** del **Dominio de la Solución**:

Tipo de Requerimiento	Descripción y Alcance	Domini o
De Negocio	Visión de más alto nivel que responde al <i>¿por qué?</i> y <i>¿para qué?</i> de la organización.	Problem a
De Usuario	Necesidades concretas de los usuarios finales en su contexto operativo.	Problem a
Funcionales	Comportamientos explícitos que el sistema debe ejecutar (equivalentes a los Casos de Uso tradicionales).	Solución
No Funcionales (RNF)	Restricciones de calidad, rendimiento, seguridad o usabilidad. Son de importancia fundamental ; su incumplimiento invalida el sistema.	Solución
De Implementación	Restricciones técnicas específicas para la construcción del software.	Solución

Historias de Usuario (*User Stories* - HU)

Una **Historia de Usuario** es una técnica ligera para capturar requerimientos expresada en lenguaje natural desde la perspectiva del usuario final. No es una especificación detallada e inmutable, sino una **intención de hacer algo** y un **mecanismo para iniciar una conversación**.

Las 3 "C" de una Historia de Usuario



1. **Card (Tarjeta):** La parte visible y sintética escrita en la tarjeta.
2. **Conversation (Conversación):** La parte más importante y no visible. Representa el diálogo continuo entre el Product Owner y el equipo técnico para descubrir los detalles del problema.

3. **Confirmation (Confirmación):** Las condiciones o criterios verificables que deben cumplirse para dar por satisfecha la historia.

Formato Estándar de Redacción

$\backslash \text{\textbf{COMO}} \} \backslash \text{\textbf{YO PUEDO}} \} \backslash \text{\textbf{Actividad/Acción}} \} \backslash \text{\textbf{DE FORMA TAL QUE}} \} \backslash \text{\textbf{Valor de Negocio}} \}$

- **Rol:** Representa el perfil o la función de quien realiza la acción (se deben evitar nombres genéricos como "Usuario" o roles intermediarios).
- **Actividad:** La acción o funcionalidad requerida del sistema.
- **Valor de Negocio:** Justifica el beneficio o la razón de ser de la actividad.

Implementación en Porciones Verticales (*Vertical Slicing*)

Las Historias de Usuario no se construyen por capas horizontales aisladas (ej. hacer sólo la base de datos o sólo la interfaz); cada historia representa una **tajada vertical completa** que atraviesa desde la interfaz de usuario (GUI), la lógica de negocio, hasta la persistencia en la base de datos.

Criterios de Aceptación y Pruebas

- **Criterios de Aceptación:** Establecen los límites de la historia, definiendo la intención del negocio sin incluir detalles de implementación técnica.
 - Indican al desarrollador **cuándo parar de codificar** y al tester **cómo diseñar las pruebas**.
 - Suelen redactarse utilizando las palabras clave: **DEBE** (obligatorio) y **PUEDE** (opcional).
- **Pruebas de Aceptación:** Ubicadas habitualmente al dorso de la tarjeta, detallan casos concretos con datos de entrada y resultados esperados (PASA / FALLA) para verificar el cumplimiento.

Criterios de Calidad: DoR, DoD e INVEST

Para asegurar la fluidez dentro del proceso de desarrollo, el equipo utiliza reglas de entrada y salida:

1. Definition of Ready (DoR) y el Modelo INVEST

El **Criterio de Ready** es un *checklist* que determina si una historia está lo suficientemente refinada para ser ingresada a un Sprint. Una Historia de Usuario cumple con el DoR si respeta el **Modelo INVEST**:

Sigla	Criterio	Significado Práctico
I	Independent (Independiente)	Se puede priorizar e implementar en cualquier orden sin depender severamente de otra historia.
N	Negotiable (Negociable)	Se negocia el <i>QUÉ</i> con el cliente, dejando abierto el <i>CÓMO</i> técnico.
V	Valuable (Valiosa)	Aporta un valor explícito y percible para el negocio o usuario.
E	Estimable (Estimable)	El equipo técnico comprende la historia lo suficiente como para calcular su esfuerzo/tamaño.
S	Small (Pequeña)	Su tamaño es adecuado para completarse holgadamente dentro de un único Sprint.
T	Testable (Comprobable)	Posee criterios claros que permiten demostrar y probar su correcto funcionamiento.

2. Definition of Done (DoD)

El **Criterio de Hecho** es el acuerdo del equipo que define las condiciones de calidad requeridas para considerar una historia totalmente terminada (ej. código compilado, pruebas unitarias y de aceptación aprobadas, sin defectos abiertos y desplegada en el entorno de pruebas).

Niveles de Abstracción de los Requerimientos

En el *Product Backlog*, las necesidades evolucionan y se desglosan progresivamente según su proximidad de ejecución:

1. **Temas:** Gran nivel de abstracción que agrupa iniciativas o propuestas de valor de alto nivel.
2. **Épicas:** Historias de usuario muy grandes ubicadas en la parte inferior de la pila. Requieren ser divididas en historias más pequeñas antes de poder implementarse.
3. **Historias de Usuario (US):** Unidades de trabajo refinadas de grano fino que cumplen con el modelo INVEST y están listas para ingresar al desarrollo.

Spikes (Historias de Investigación)

Un **Spike** es un tipo especial de Historia de Usuario orientada exclusivamente a **reducir el riesgo o la incertidumbre** técnica o funcional de otra historia del backlog.

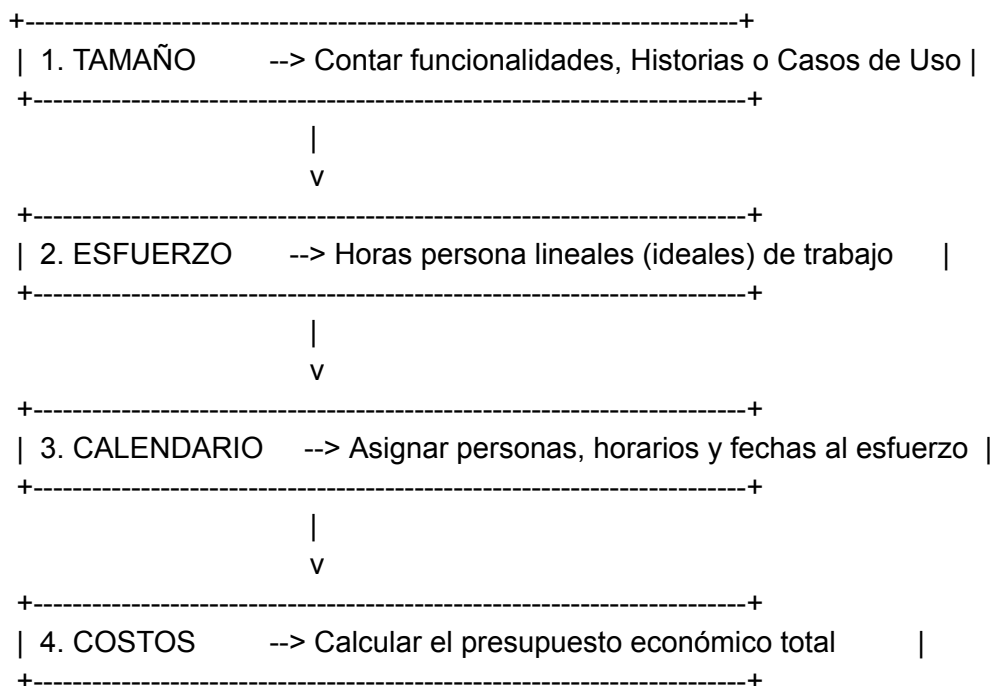
- **Spike Técnico:** Investigar la viabilidad de una tecnología, arquitectura, librería o integraciones desconocidas en el dominio de la solución.
 - **Spike Funcional:** Evaluar la interacción del usuario con el sistema mediante prototipos para aclarar comportamientos dudosos.
 - **Reglas de Aplicación:** Se ejecutan habitualmente en el Sprint previo al desarrollo de la historia que tenía la incertidumbre. Deben ser estimables, demostrables, estar acotadas en tiempo (*time-boxed*) y utilizarse como **última opción** cuando la duda no pueda resolverse en la conversación habitual.
-

Naturaleza de la Estimación de Software

- **Definición:** Estimar es predecir el comportamiento del desarrollo en un momento donde no se tiene conocimiento completo y existe alta incertidumbre.
 - **Evolución de la Incertidumbre:** La incertidumbre es máxima al inicio del proyecto. A medida que el desarrollo avanza y se obtiene más información, las estimaciones se deben ajustar y refinarse continuamente para volverse más precisas.
 - **Estimar vs. Planear: Estimar NO es planear.** La estimación sirve como insumo o base para el plan, pero los planes incorporan decisiones de negocio y compromisos. Las estimaciones no son compromisos; utilizarlas como promesas rígidas resulta peligroso y perjudicial para la organización.
 - **Causas Frecuentes de Error al Estimar:**
 - Desorganización del proyecto y bajo conocimiento del producto.
 - Desconocimiento de las capacidades reales del equipo o de los recursos disponibles.
 - Exceso de optimismo habitual en los desarrolladores y subestimación de dificultades.
 - Omitir actividades esenciales como tareas administrativas, reuniones, reuniones de compañía, revisiones o creación de datos para pruebas.
 - Utilizar una única técnica de estimación sin contrastar resultados.
-

Secuencia Lógica de la Estimación

La gestión de proyectos exige estimar variables en un orden de dependencia específico:



1. **Tamaño:** Medida de qué tan grande es el trabajo mediante técnicas de conteo (contar requerimientos, Historias de Usuario o Casos de Uso) o comparación con datos históricos.
 2. **Esfuerzo:** Cantidad de horas lineales o ideales necesarias para construir el software, sin considerar paralelismo, interrupciones o quién lo ejecuta.
 3. **Calendario / Tiempo:** Distribución del esfuerzo estimado en días y horas de trabajo asignando personal concreto.
 4. **Costos y Presupuesto:** Determinación monetaria derivada de las estimaciones anteriores.
-

Clasificación de Técnicas de Estimación

Las técnicas de estimación se dividen en diferentes métodos según la fuente utilizada:

- **Basados en la Experiencia:**
 - **Juicio de Experto Puro:** Un experto analiza las especificaciones y proporciona un número. Su riesgo es la alta subjetividad y la dependencia de la persona.
 - **Wideband Delphi:** Un grupo de personas estima de forma anónima e individual; luego se comparten las estimaciones y se realizan rondas sucesivas de discusión hasta lograr convergencia. Genera mejores resultados que el juicio individual.
 - **Estimación PERT (Optimista, Pesimista y Habitual):** Estructura el juicio de experto aplicando la fórmula:
$$\text{EsfuerzoEstimado} = \frac{O+4H+P}{6}$$
 -
 - **Analogía / Datos Históricos:** Compara el proyecto actual con proyectos anteriores similares almacenados en la organización.
 - **Basados en Recursos:** Determinan el cálculo en función de la disponibilidad y costo de recursos.
 - **Basados en Mercado:** Miden el costo en función de proyectos similares comercializados externamente.
 - **Algorítmicos:** Aplican modelos matemáticos o estadísticos (ej. COCOMO) considerando variables de tamaño, productividad y complejidad.
-

Estimación en Ambientes Ágiles: Story Points (SP)

En metodologías empíricas y ágiles, la estimación del tamaño se desvincula de las horas absolutas y del tiempo calendario para enfocarse en la **estimación relativa**.

- **Story Points (SP):** Unidad de medida de tamaño relativo asignada a una *feature* u Historia de Usuario. Representa una ponderación combinada de tres factores:

$$\text{StoryPoints} = \text{Complejidad} + \text{Esfuerzo} + \text{Incertidumbre/Riesgo}$$

- **Propiedades de los Story Points:**
 - **Medida Relativa y del Equipo:** Cada equipo define su propia escala comparando historias contra una **Historia de Usuario Canónica (o base)**. Los Story Points de un equipo **NO son comparables** con los de otro equipo.
 - **Separación de Tamaño y Tiempo:** Desconecta el tamaño del software de la duración del proyecto, evitando los errores causados por diferencias individuales de habilidad o experiencia técnica.
-

Métrica de Velocidad (*Velocity*)

La **Velocidad** es la métrica clave que mide el progreso real de un equipo en un marco iterativo.

- **Cálculo de la Velocidad:** Es la **suma de los Story Points** correspondientes a las Historias de Usuario que fueron completadas **al 100%** durante la iteración.
- **Regla Estricta:** No se acreditan ni suman Story Points por historias parcialmente completadas.
- **Proyección de la Duración del Proyecto:** A partir de la velocidad promedio observada, se calcula el número de Sprints necesarios para completar el *Product Backlog*:

$$\text{Cantidad de Sprints} = \frac{\text{Total de Story Points del Product Backlog}}{\text{Velocidad Promedio del Equipo}}$$

Dinámica de Estimación: Planning Poker

El **Planning Poker** es la técnica colaborativa más difundida en agilidad. Combina el Juicio de Experto, la Analogía y principios de Wideband Delphi.

1. Presentación de la US por el Product Owner
 2. Diálogo y aclaración de dudas técnicas
 3. Selección individual e incógnita de cartas
 4. Revelación simultánea de votos
 5. Debate entre estimaciones extremas (mínima y máxima)
 6. Re-votación hasta alcanzar consenso
- **Uso de la Serie de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89...):** Se utilizan cartas numeradas con la secuencia de Fibonacci porque su incremento exponencial refleja la incertidumbre creciente a medida que el tamaño de la historia aumenta.
 - **Reglas de Puntuación:**

- **Historia Canónica:** Se le asigna habitualmente un valor bajo de referencia (ej. 1 o 2 SP).
 - **Valor 0:** Representa que la historia no requiere esfuerzo significativo o que existe un desconocimiento que exige refinarla antes.
 - **Límite de Tamaño (Historias Grandes):** Un valor de Story Points demasiado alto (ej. mayor a 8 SP) indica que la historia es muy compleja o amplia, por lo que **no cumple el Criterio de Ready (INVEST: *Small*)** y debe dividirse en historias más pequeñas.
 - **Participantes:** Es una actividad ejecutada exclusivamente por el **equipo de desarrollo**, ya que quienes estiman deben ser las personas técnicamente competentes para construir la solución.
-

Scrum es un marco de trabajo (*framework*) ligero que ayuda a personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos. No es una metodología rígida ni un proceso prescriptivo, sino una estructura basada en el empirismo y el pensamiento Lean.

1. Fundamentos: Empirismo y Valores de Scrum

Los 3 Pilares del Empirismo

Scrum sostiene que el conocimiento proviene de la experiencia y la toma de decisiones se basa en lo que se observa y mide en la realidad:

1. **Transparencia:** El proceso y el trabajo deben ser visibles tanto para quienes realizan el trabajo como para quienes lo reciben.
2. **Inspección:** Los artefactos y el progreso hacia los objetivos deben inspeccionarse con frecuencia para detectar desviaciones o problemas no deseados.
3. **Adaptación:** Si un aspecto del proceso se desvía de los límites aceptables, el proceso o el material que se está procesando debe ajustarse lo antes posible para minimizar desviaciones futuras.

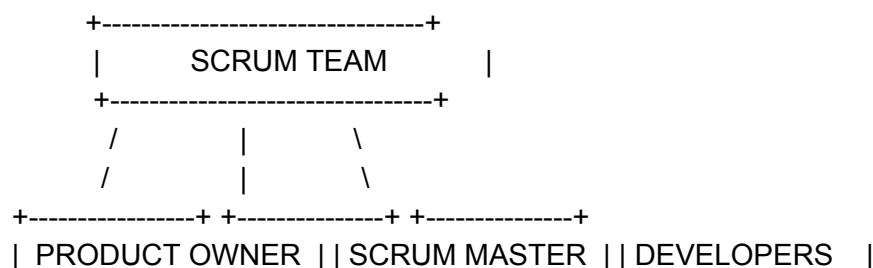
Los 5 Valores de Scrum

Para que el empirismo funcione, el equipo debe vivir y aplicar cinco valores fundamentales:

- **Compromiso (*Commitment*):** Dedicarse a alcanzar las metas del equipo.
 - **Foco (*Focus*):** Concentrarse en el trabajo del Sprint y en los objetivos seleccionados.
 - **Apertura (*Openness*):** Transparencia sobre el trabajo y los desafíos del desarrollo.
 - **Respeto (*Respect*):** Confiar en la capacidad e independencia de los miembros del equipo.
 - **Coraje (*Courage*):** Valor para hacer lo correcto y abordar problemas complejos.
-

2. El Equipo Scrum (*Scrum Team*)

El *Scrum Team* es pequeño (típicamente de 3 a 10 personas), multifuncional y **auto-organizado / autogestionado**. No existe la figura del *Project Manager* tradicional dentro del equipo:



+-----+ +-----+ +-----+

- **Product Owner (PO):**

- Responsable de **maximizar el valor del producto** resultante del trabajo del equipo.
- Es el único encargado de gestionar de forma exclusiva el **Product Backlog** (ordenar, priorizar y expresar claramente los elementos).
- Representa las necesidades del negocio y los usuarios frente al equipo técnico.

- **Scrum Master (SM):**

- Responsable de **promover y apoyar Scrum** guiando al equipo y a la organización en la comprensión de la teoría y práctica del marco.
- Funciona como un **líder que sirve** (*servant leader*), removiendo impedimentos que bloquean el avance del equipo y facilitando los eventos de Scrum.

- **Desarrolladores (Developers):**

- Personas del equipo comprometidas a crear cualquier aspecto de un **Incremento utilizable** en cada Sprint.
- Tienen la autonomía para decidir el "**cómo**" técnico de la solución y son los únicos responsables de estimar el esfuerzo del *Backlog*.

3. Los 3 Artefactos de Scrum y sus Compromisos

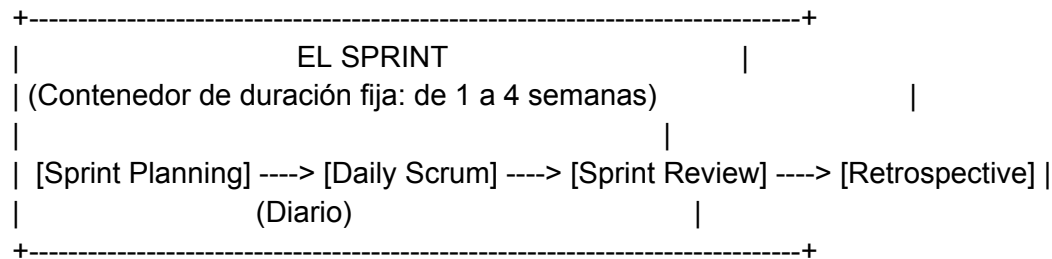
Los artefactos representan trabajo o valor y están diseñados para maximizar la transparencia. Cada artefacto incluye un compromiso explícito para medir el progreso:

Artefacto	Descripción	Compromiso Asociado
Product Backlog	Lista ordenada y viva de todo lo que se conoce que es necesario en el producto.	Objetivo del Producto (<i>Product Goal</i>): Describe un estado futuro del producto que sirve como meta de largo plazo.
Sprint Backlog	Conjunto de elementos del Product Backlog seleccionados para el Sprint, más el plan para entregarlos.	Objetivo del Sprint (<i>Sprint Goal</i>): Propósito único fijado para el Sprint que brinda flexibilidad sobre el alcance exacto.
Incremento	La suma de todos los elementos del Product Backlog completados durante el Sprint, integrados con los incrementos anteriores.	Definición de Hecho (<i>Definition of Done - DoD</i>): Criterio formal de calidad que debe cumplir cada

elemento para considerarse terminado.

4. Los 5 Eventos de Scrum (*Time-boxed*)

Todos los eventos de Scrum tienen un límite de tiempo fijo (*time-box*) para crear regularidad y evitar reuniones innecesarias:



1. El Sprint (Contenedor):

- Bloque de tiempo de duración fija (habitualmente de 1 a 4 semanas) donde se crea un Incremento de producto funcional y de valor.
- Durante el Sprint no se realizan cambios que pongan en riesgo el Objetivo del Sprint.

2. Sprint Planning (Planificación del Sprint):

- Evento al inicio del Sprint donde todo el equipo Scrum colabora para responder: *¿Por qué es valioso este Sprint?, ¿Qué se puede hacer? y ¿Cómo se realizará el trabajo elegido?*

3. Daily Scrum (Daily / Reunión Diaria):

- Evento de **15 minutos** diario para que los desarrolladores inspeccionen el progreso hacia el Objetivo del Sprint y adapten el plan de trabajo para las próximas 24 horas.

4. Sprint Review (Revisión del Sprint):

- Se realiza al final del Sprint para inspeccionar el Incremento construido junto con las partes interesadas (*stakeholders*) y adaptar el Product Backlog según la retroalimentación recibida.

5. Sprint Retrospective (Retrospectiva del Sprint):

- Evento de cierre donde el equipo Scrum se enfoca en la **mejora continua** (*Kaizen*), inspeccionando cómo fue el último Sprint respecto a las personas, las relaciones, los procesos y las herramientas, para definir acciones concretas de mejora.